DATAOPS AGENT

QueryPilot

Agente de IA para consultar dados em linguagem natural, com segurança e rastreabilidade

João Pedro Wiskow · Matheus Melo · Enzo Picoli

Sumário

01 **Problemática e objetivo**

02 **Segunda** Kickoff e banco de dados

03 **Terça** Ferramentas de dados e Function Calling

04 **Quarta** Guardrails, MCP e agente ReAct

05 **Quinta** Interface web e explicabilidade

06 **Sexta** Cache, resiliência e Demo Day

07 **Conclusão e testagem**

Problemática e objetivo

Problemática

Consultar dados exige SQL, bases reais trazem anomalias (nulos, duplicados, valores negativos) e deixar uma IA gerar consultas abre risco de injeção de SQL.

Objetivo

Construir o QueryPilot: um agente que responde perguntas de negócio em linguagem natural, de forma segura, explicável e resiliente.

Dia 16 · 05/10 · Segunda-feira

J João Pedro · E Enzo · M Matheus

Segunda: Kickoff do projeto

Definir o domínio de dados, modelar um SQLite com anomalias propositais, estruturar o repositório Git e validar a base com Smoke Test Cruzado.

`docs/dicionario_dados.md`	Tabelas, relacionamentos, perguntas de negócio e anomalias	J E M
`src/database/init_db.py`	Cria o SQLite, FKs ativas, índices e reset	J E
`src/database/seed_data.py`	Povoamento determinístico com anomalias injetadas	E
`data/dataops.db`	Banco SQLite gerado	E
`data/.gitkeep`	Mantém a pasta data/ no Git	J
`tests/smoke_test_db.py`	Integridade, contagem, JOINs e anomalias	M
`.gitignore`	Protege segredos e o banco local	J
`requirements.txt`	Dependências do projeto	J
`README.md`	Execução e escala Piloto/Copiloto	J

Testagem do conjunto J E M

Dia 17 · 06/10 · Terça-feira

J João Pedro · E Enzo · M Matheus

Terça: Ferramentas e Function Calling

Implementar ferramentas de schema, profiling e execução controlada de SQL, com testes preliminares de Function Calling no LLM.

`src/tools/schema_tools.py`	Tabelas, schemas e chaves estrangeiras via PRAGMA, com validação	M
`src/tools/profiling_tools.py`	Nulos, distintos, estatísticas descritivas e amostras	J
`src/tools/query_tools.py`	SQL somente leitura, LIMIT 50, tempo de execução e JSON	E
`src/agent/test_tools_llm.py`	7 ferramentas + gemini-3.8-flash via Function Calling	J
`__init__.py`	Em src/, src/database/, src/tools/, src/agent/ e src/mcp_server/	J E M

Testagem do conjunto J E M

Dia 18 · 07/10 · Quarta-feira

J João Pedro · E Enzo · M Matheus

Quarta: Segurança, MCP e agente ReAct

Guardrails determinísticos contra SQL perigoso, servidor FastMCP e loop ReAct com auto-recuperação de erros, validado por testes adversariais.

`src/agent/guardrails.py`	Só SELECT/WITH; bloqueia DROP, DELETE, PRAGMA e múltiplos comandos	M
`src/mcp_server/dataops_mcp.py`	Servidor FastMCP com ferramentas de dados e guardrail	J
`src/agent/dataops_agent.py`	Agente ReAct com auto-recuperação de erros	E
`tests/test_guardrails_attacks.py`	Red teaming: injeções SQL e jailbreak, zero alterações no banco	M
`.env`	Variáveis de ambiente e chaves, fora do Git	J M

Testagem do conjunto J E M

Dia 19 · 08/10 · Quinta-feira

J João Pedro · E Enzo · M Matheus

Quinta: Interface web e explicabilidade

Desenvolver a interface em Streamlit com painel Glass-Box AI, métricas da base e gráficos dinâmicos, além do ensaio do pitch para o Demo Day.

`app.py`	Chat com session_state, métricas do SQLite e reset de contexto	E
`renderizar_trace / renderizar_dados`	Painel de auditoria: ferramentas, guardrail, SQL, tempo, tabela e gráfico	E M

Testagem do conjunto — J M

Dia 20 · 09/10 · Sexta-feira

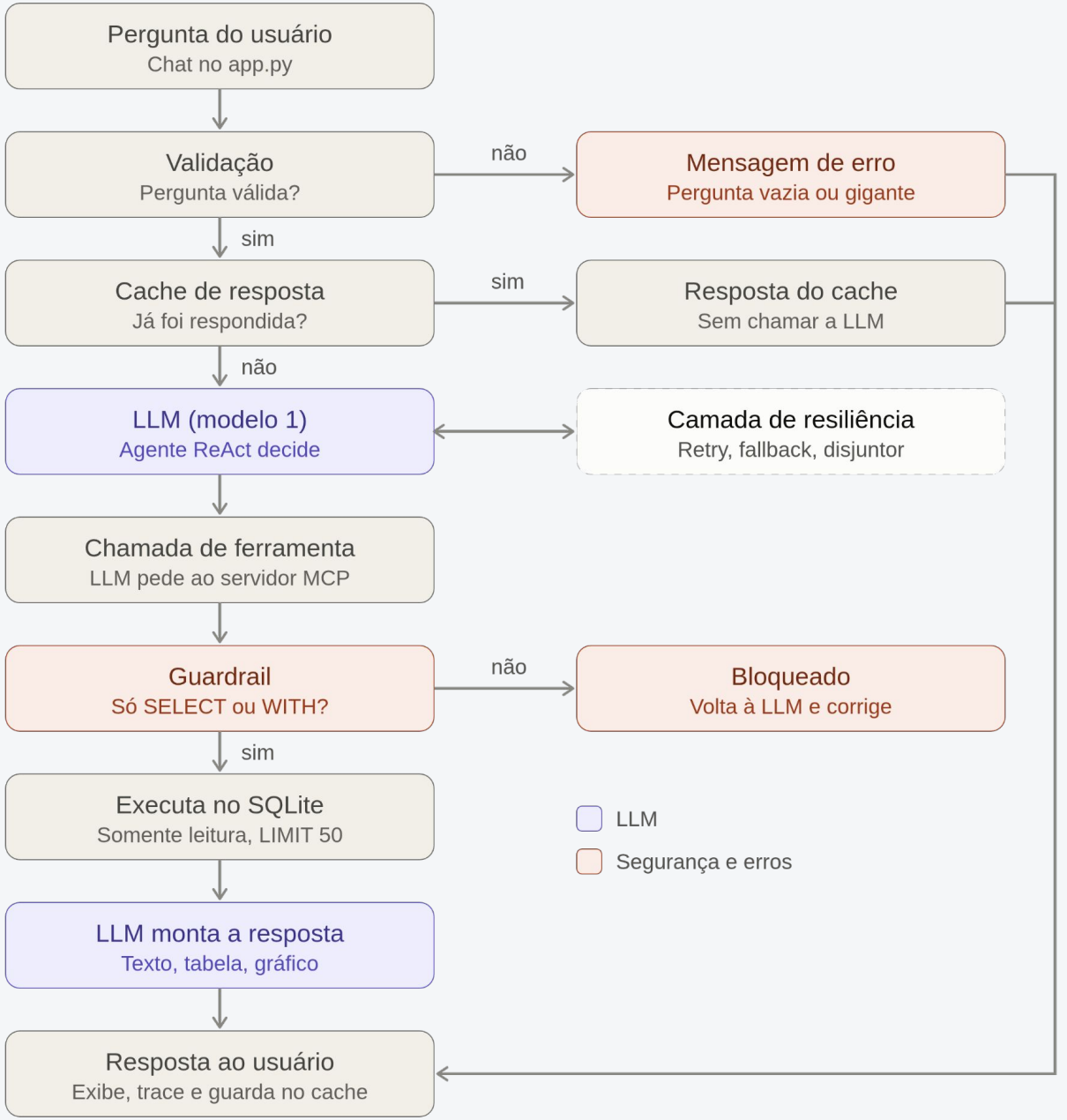
J João Pedro · E Enzo · M Matheus

Sexta: Cache, resiliência e Demo Day

Cache inteligente, fallback entre modelos LLM, interface final para o Demo Day e novas ferramentas de profiling no servidor MCP.

`src/agent/cache.py`	Cache em memória com invalidação ao alterar o banco	J
`src/agent/resiliencia.py`	Retries, fallback entre modelos e circuit breaker de 60s	J
`app.py (reescrito)`	Interface do Demo Day: atalhos, KPIs, gráficos e CSV	J
`src/mcp_server/dataops_mcp.py (atualizado)`	Novas ferramentas de profiling no catálogo MCP	J
`docs/MELHORIAS.md`	Documento de melhorias do projeto	J

Testagem do conjunto J


Pergunta do usuário
Chat no app.py
Validação
Pergunta válida?
não
Mensagem de erro
Pergunta vazia ou gigante
sim
Cache de resposta
Já foi respondida?
sim
Resposta do cache
Sem chamar a LLM
não
LLM (modelo 1)
Agente ReAct decide
Camada de resiliência
Retry, fallback, disjuntor
Chamada de ferramenta
LLM pede ao servidor MCP
Guardrail
Só SELECT ou WITH?
não
Bloqueado
Volta à LLM e corrige
sim
Executa no SQLite
Somente leitura, LIMIT 50
LLM monta a resposta
Texto, tabela, gráfico
Resposta ao usuário
Exibe, trace e guarda no cache
LLM
Segurança e erros

Conclusão e testagem

Testagem

Resiliência

Entrega

DATAOPS AGENT

QueryPilot

Agente de IA para consultar dados em linguagem natural, com segurança e rastreabilidade

João Pedro Wiskow · Matheus Melo · Enzo Picoli